

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: Sep 15, 2026	DATE SUBMITTED: Sep 15, 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 9.1	Implementing Trees
1. PROCEDURE OUTPUT	
What procedures did you perform?	
1. Code Snippet • Submit the complete code for your <code>BinaryTree</code> class and its <code>main.cpp</code> file. • The code must demonstrate the creation and basic manipulation of a Binary Tree.	

- Submit the complete code for your `BinaryTree` class and its `main.cpp` file.
- The code must demonstrate the creation and basic manipulation of a Binary Tree.

Header

```
1 > //...
4
5 #ifndef UNTITLED1_HEADER_H
6 #define UNTITLED1_HEADER_H
7
8 v #include <iostream>
9 #include <string>
10
11 template <typename T>
12 v class BinaryTree {
13 private:
14     T key; // Stores the data/value of the node
15     BinaryTree<T> *leftChild; // Pointer to the left child subtree/node
16     BinaryTree<T> *rightChild; // Pointer to the right child subtree/node
17
18 public:
19     // Constructor
20     BinaryTree(T rootObj);
21
22     // Destructor
23     ~BinaryTree();
24
25     // Insert childrens - Pushes any existing child down and inserts the new node above it
26     void insertLeft(T newNode);
27     void insertRight(T newNode);
28
29     // Accessing the child
30     BinaryTree<T> *getLeftChild();
31     BinaryTree<T> *getRightChild();
32
33     // Get and set node value
34     void setRootVal(T obj);
35     T getRootVal();
36
37     // Checker for leaf
38     bool isLeaf();
39 };
```

```

41     template <typename T>
42     BinaryTree<T>::BinaryTree(T rootObj) {
43         key = rootObj;
44         leftChild = nullptr;
45         rightChild = nullptr;
46     }
47
48     // Destructor implementation to clean up heap memory
49     template <typename T>
50     BinaryTree<T>::~BinaryTree() {
51         delete leftChild;
52         delete rightChild;
53     }
54
55     // Inserting left: Pushes existing left child down if one already exists
56     template <typename T>
57     void BinaryTree<T>::insertleft(T newNode) {
58         if (leftChild == nullptr) {
59             leftChild = new BinaryTree<T>(newNode);
60         } else {
61             BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
62             newChild->leftChild = leftChild; // Moves the existing left child below the new node
63             leftChild = newChild;           // Makes the new node the left child
64         }
65     }
66
67     // Inserting right: Pushes existing right child down if one already exists
68     template <typename T>
69     void BinaryTree<T>::insertright(T newNode) {
70         if (rightChild == nullptr) {
71             rightChild = new BinaryTree<T>(newNode);
72         } else {
73             BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
74             newChild->rightChild = rightChild; // Moves the existing right child below the new node
75             rightChild = newChild;           // Makes the new node the right child
76         }
77     }

```

```

76     }
77 }
78
79 // Accessing the left child pointer
80 template <typename T>
81 → ↵ BinaryTree<T>* BinaryTree<T>::getLeftChild() {
82     return leftChild;
83 }
84
85 // Accessing the right child pointer
86 template <typename T>
87 → ↵ BinaryTree<T>* BinaryTree<T>::getRightChild() {
88     return rightChild;
89 }
90
91 // Setting the value of the node's key
92 template <typename T>
93 → ↵ void BinaryTree<T>::setRootVal(T obj) {
94     key = obj;
95 }
96
97 // Getting the value of the node's key
98 template <typename T>
99 → ↵ T BinaryTree<T>::getRootVal() {
100     return key;
101 }
102
103 // Checking if the node is a leaf (has no left or right children)
104 template <typename T>
105 → ↵ bool BinaryTree<T>::isLeaf() {
106     return (leftChild == nullptr && rightChild == nullptr);
107 }
108
109 #endif // UNTITLED1_HEADER_H

```

Main

```
1 //  
2 // Created by royga on 9/15/2026.  
3 //  
4 #include <iostream>  
5 #include "header .h"  
6  
7  
8  
9 int main() {  
10     // Create the root node 'A'  
11     BinaryTree<char>* root = new BinaryTree<char>(rootObj: 'A');  
12  
13     // Create children of A (using lowercase insertleft / insertright)  
14     root->insertleft(newNode: 'B');  
15     root->insertright(newNode: 'C');  
16  
17     // Create children of B  
18     root->getLeftChild()->insertleft(newNode: 'D');  
19     root->getLeftChild()->insertright(newNode: 'E');  
20  
21     // Create children of C (C has 'F' on the left, nullptr on the right)  
22     root->getRightChild()->insertleft(newNode: 'F');  
23  
24     // Verification output  
25     std::cout << "Root: " << root->getRootVal() << '\n';  
26     std::cout << "Left child of A: " << root->getLeftChild()->getRootVal() << '\n';  
27     std::cout << "Right child of A: " << root->getRightChild()->getRootVal() << '\n';  
28     std::cout << "Left child of B: " << root->getLeftChild()->getLeftChild()->getRootVal() << '\n';  
29     std::cout << "Right child of B: " << root->getLeftChild()->getRightChild()->getRootVal() << '\n';  
30     std::cout << "Left child of C: " << root->getRightChild()->getLeftChild()->getRootVal() << '\n';  
31  
32     // Optional: Clean up heap memory if you don't rely solely on scope  
33     delete root;  
34  
35     return 0;  
36 }
```

2. Documentation

- Submit a document explaining the methods/functions of the `BinaryTree` class you created.
- Use your Activity Template 2026

Methods and Functions:

BinaryTree(T rootObj) - this initializes a new tree node with a specific data value and sets both the children in the left and right as nullptr.

~BinaryTree() - this clear all dynamically allocated nodes from the heap.

insertleft(T newNode) - add a new node to the left branch. And if there is already a child, it makes a new node above the existing child, and pushes the subtree one level down.

insertright(T newNode) - add a new node to the right branch. And if there is already a child, it makes a new

node above the existing child, and pushes the subtree one level down.

getLeftChild() - returns a pointer referencing the left child node.

getRightChild() - returns a pointer referencing the right child node.

setRootVal(T obj) - updates or modifies the data key stored in the current node.

getRootVal() - returns the data value stored in the current node.

isLeaf() - evaluates whether the current node is a leaf.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.

